AI6125-MAS GROUP 7 PROJECT REPORT *

Deng Zhouheng
G2504130B
zhuoheng001@e.ntu.edu.sg

Lou Enge
<G2504803B>
enge001@e.ntu.edu.sg

Sun Qingmo
<Matric Number>
qingmo001@e.ntu.edu.sg

Wu Shanni
<Matric Number>
swu029@e.ntu.edu.sg

Xiong Maihe
<Matric Number>
maihe001@e.ntu.edu.sg

Zhang Yanhai
G2509626L
zh0010ai@e.ntu.edu.sg

ABSTRACT

This report presents Group 7's six-agent system for Tileworld, a partially observable and dynamic grid environment with fuel, sensing, and carrying constraints. Our implementation combines a shared engineering layer (`Group7AgentBase`), protocol-driven collaboration (G7P2), and agent-specific planning policies. We evaluate the system on three parameter sets and report mean reward over repeated runs. Results and ablations show that both communication and policy specialization are necessary for strong team performance, especially in larger and more dynamic maps.

1 Introduction & Background

Tileworld is a standard benchmark for studying multi-agent decision making under uncertainty [1, 2]. Agents operate on a grid with partial observability, stochastic object dynamics, limited fuel, and finite carrying capacity; hence effective behavior requires both local intelligence and team coordination.

In this project, six agents jointly act in the same environment using a Sense→Communicate→Plan→Act loop. We focus on three design questions: (i) how to share local observations efficiently, (ii) how to reduce conflict and duplicated effort, and (iii) how much performance comes from shared infrastructure versus agent-specific policies. The rest of this report presents our implementation, experimental settings, quantitative results, and ablation findings.

2 Environment & Experiment Setup

Our experiment is conducted in a tileworld simulator implemented in Java 1.8 with the Java3D v1.5.1 and MASON 14 package. One simulation will run for 5000 steps and we take the final reward as the metric of the agents system; different simulations will start with a randomly generated seed. The reward is defined by the number of holes filled. The initial fuel is fixed at the maximum fuel carriage of 500.

There are 3 different configurations that affect world size, object lifetime, and object creation distribution.
- Parameter Set 1: World Size = 50×50, Object Creation Rate $\sim\mathcal{N}(\mu = 0.2, \sigma = 0.05)$, Lifetime = 100.
- Parameter Set 2: World Size = 80×80, Object Creation Rate $\sim\mathcal{N}(\mu = 2, \sigma = 0.5)$, Lifetime = 30.
- Parameter Set 3: World Size = 120×120, Object Creation Rate $\sim\mathcal{N}(\mu = 5, \sigma = 0.5)$, Lifetime = 50.

3 Agent Design

3.1 Group7AgentBase - Abstract Base Class

We centralized basic Agent behaviors to this base agent class that extends the `TWAgent` class.

* *Contributions*: **Authors are listed alphabetically, all authors contributed to this project equally.**

Group7AgentBase is a reusable control and coordination layer for all Group 7 agents. It standardizes low-level mobility with A* path planning, path reuse/clearing, obstacle-aware stepping, and a deterministic zig-zag fallback for exploration when no high-level target is available. It also provides fuel-station handling so subclasses can focus on policy rather than survival plumbing. On communication, the base defines the G7P2 protocol and parser, supports directed and broadcast messages, and auto-emits basic messages like discovery and sensor snapshots.

3.2 AgentHanny - The Proactive Learner

This Agent is designed and implemented by Zhang Yanhai, as an agent that dynamically modeling the current environment with observation data gathered from the team and utilizes the estimation in calculating the expected utility in the planning phase.

Planning This agent decides its action based on **utility over plan**, where a plan is an action sequence from the current position. To determine this, we estimate object survival probability and aggregate utilities over all targets in the sequence. Survival analysis [3] was used for estimating the object existing probability and the full derivation is included in the appendix.

AgentHanny will form plans from memory with picking up 1...available-carry-space tiles and filling them to 1...expected-carried-tiles holes, then calculate the expected utility (details in the appendix). Then the plan with highest EU will be selected and executed, with Target Lock send to teammates. The agent action flow can be summarized as follows:

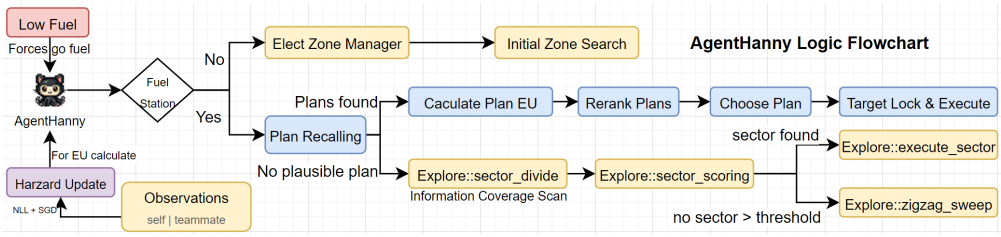


Figure 1: AgentHanny Action Flow

Communications Agent Hanny follows the protocol defined in `Group7AgentBase` and consumes team messages each think cycle. It uses: Target lock is acquired/kept for the *current execution target* (not for every recalled target at plan creation time), renewed before expiry, and released when target is completed, invalidated, pre-empted, or when mode switches (e.g., refuel/re-plan).

Memory Layer In order to make the online MLE possible, AgentHanny keeps a memory side-card to record additional information that `TWAgentWorkingMemory` does not provide. Additional information includes objectType, firstSeen, lastSeen, etc. This side card is always synchronized with the original working memory, and the information is updated when new observation is made.

Exploration In order to maximize the social welfare, AgentHanny is designed to explore the sections of the tileworld map that are least explored by the team. To be more specific, AgentHanny divides the map into a dynamic $n \times n$ grid, where n is set to $WorldSize//15$. Each section is scored with negative signals from teammate density, explored coverage, and distance from current position. Sections whose center has a too-long distance from AgentHanny is excluded. The section with the best score is selected, and if no valid section exists, the agent falls back to zig-zag exploration.

3.3 AgentEnge – The Priority-Driven Planner

This agent is designed and implemented by Lou Enge as an agent that operates via a strict four-level priority state machine, combining obstacle-aware A* path planning with a lawnmower exploration strategy and G7P2-compatible target-lock conflict avoidance.

Planning AgentEnge makes decisions through a prioritised cascade evaluated every think cycle:

1. **Survival** (fuel < 180): all other goals are suspended; the agent navigates to the nearest known fuel station, re-discovering it through lawnmower scan if not yet in memory.

2. **Fill-Hole** (carrying ≥ 1 tile and a hole is in memory): the agent selects a fill target by iterating holes in memory sorted by Manhattan distance, skipping any hole that has already disappeared from the environment grid or is held under a higher-priority teammate lock, then navigates to the first remaining candidate.
3. **Pickup** (carried tiles < 3 and a tile is in sensor range): acquire a target lock and navigate to the nearest visible tile.
4. **Exploration**: lawnmower scan of the assigned zone.

Path planning uses an internal A* implementation with Manhattan-distance heuristic. Before committing to a path, the agent evaluates feasibility against two criteria: remaining fuel must exceed path length, and for objects with finite lifetimes the estimated travel steps must not exceed the object's remaining time-to-live. If A* fails or the cost check rejects the path, the agent refreshes its obstacle memory from the current sensor view and retries up to $k = 3$ times before abandoning the target.

Communications AgentEnge adheres to the G7P2 protocol and processes all cooperative messages each think cycle (see Section 4 for protocol details). Its primary contribution to the conflict-avoidance layer is issuing `TL` messages upon entering Pickup or Fill-Hole mode, renewed each cycle with a TTL of $[8, 20]$ steps and an inverse-distance priority score. A higher-priority teammate lock immediately pre-empts the local target, redirecting the agent to exploration without a wasted trip.

Memory & Obstacle Handling AgentEnge maintains two layers of position history to suppress oscillation: a short deque (depth 3) for immediate back-tracking avoidance and a longer deque (depth 8) for two-cell-cycle detection ($A \rightarrow B \rightarrow A \rightarrow B$ patterns). When either pattern is detected the lawnmower cursor is forcibly advanced and the agent takes one unconstrained step to escape. Because `TWFuelStation` is not a `TWObject` and is therefore not automatically stored by the framework's working memory, the agent manually scans its sensor footprint each step and writes any observed fuel station directly into the memory grid.

Exploration The lawnmower scan partitions the map into rows of height equal to the sensor diameter ($2r + 1 = 7$ cells), sweeping left-to-right on even passes and right-to-left on odd passes with a step size of 7. Each agent's starting cursor is offset by a hash of its name to prevent six agents from scanning in lockstep at startup. During the zone-bootstrap phase the scan is clipped to the agent's ZA-assigned rectangle; after coordination completes, soft repulsion from teammate SS positions nudges waypoints away from dense clusters to reduce sensor overlap. If the computed waypoint is already occupied by the agent, the cursor advances immediately to avoid a null-path spin.

3.4 Agent 3 - <SmartAgent / The Explorer>

The SmartAgent is a collaborative agent designed for the TileWorld simulation environment. Its core objective is to expand the environmental field of vision through efficient edge-exploration strategies and share key information discovered, such as the location of gas stations, tiles, and holes, with teammates to provide vision support for team.

Core Design Principles The design of intelligent agents revolves around three core principles: safe exploration, controllable fuel, and collaborative sharing. The collaboration protocol follows our communication design.

Think Principles The agent's decision-making logic follows a clear priority hierarchy. When no gas station is found, a serpentine grid search strategy is initiated to locate a gas station. When fuel is below the refueling threshold (150), it prioritizes refueling at known gas stations, when fuel is at the warning threshold (200), it only performs short-range tasks, and the movement is restricted to approaching gas stations. When there are no interactive tasks and fuel is safe, the core edge-exploration logic is executed to maximize the coverage of unknown areas.

Exploration strategy Prioritizing moving along walls or holes, utilizing edge area exploration capabilities, which allows for faster coverage of unknown areas compared to random wandering, improving the efficiency of expanding your field of vision. Path planning prioritizes the limited A* algorithm, switches to the greedy algorithm when A* fails, and finally uses short-depth BFS as a fallback.

3.5 Agent 4 - <Role / Name>

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut

leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

3.6 AgentDeng - <Utility-Driven Agent>

Decision Architecture AgentDeng is designed and implemented by Deng Zhouheng, adopting a utility-driven planner instead of a fixed-priority state machine to achieve flexible, context-aware decision making. At each step, four candidates — fuel station, hole, tile, and exploration — are scored simultaneously and the highest is selected:

$$\begin{aligned} U_{\text{refuel}} &= 200 \cdot e^{-4m/60}/(1+d), \\ U_{\text{hole}} &= 50f/(1+d), \\ U_{\text{tile}} &= 30f/(1+d), \\ U_{\text{explore}} &= 8/(1+d), \end{aligned}$$

where $f = t_{\text{left}}/(t_{\text{left}} + d)$ and m is the fuel margin. The exponential-decay fuel model avoids both premature refueling and late-game starvation. Replanning is triggered only when the target disappears or a candidate exceeds the current utility by $1.3\times$, preventing oscillation.

Memory and Exploration The agent maintains the standard `ObjectGrid2D` memory and uses object freshness to filter out candidates that cannot be reached before decay. Stale entries within sensor range are periodically verified against the real environment and removed once the corresponding objects vanish. A `visited[][]` bitmap tracks sensor-covered cells and guides exploration toward the nearest unvisited region. During the bootstrap phase, the agent performs a snake-pattern sweep within its assigned zone to efficiently locate the fuel station.

Anti-Stuck Fallback A three-level mechanism handles pathfinding failures: (1) blacklist the unreachable target for 20 steps, (2) fall back to a greedy one-step move, (3) after five stationary steps, purge all remembered obstacles and replan. This robustness, combined with the utility planner, achieved a higher reward.

3.7 Agent 6 - <Role / Name>

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

4 Agent Communication & Collaboration

Communication is implemented in `Group7AgentBase` under protocol version `G7P2`, with message tuple $\langle version, step, from, to, type, x, y, payload \rangle$. In `TWEnvironment`, `sense()` and `communicate()` are scheduled before `think()` and `act()`, so messages produced in a step are readable by teammates in that same decision cycle.

The protocol combines sparse events and dense snapshots. Event messages broadcast newly observed objects (`NT/NH/OB`), one-time fuel-station discovery (`FS`), and action confirmations (`PK/FH`). Snapshot message `SS` encodes local empty/occupied cells each step to refresh shared belief quickly. Collaboration is staged: bootstrap zone assignment uses `ZA/ZK` (deterministic manager election with bounded retries), while runtime conflict avoidance uses `TL/TR` target locks with priority and TTL. Together, these mechanisms reduce redundant pursuit and improve task allocation stability under partial observability. And the use of target management enpowers our agents to form a expanding area of exploration and exploitation, illustrating as the figure below.

So that we may minimize step wasted on unnecessary movements $\notin$ {plan execution, exploration}.

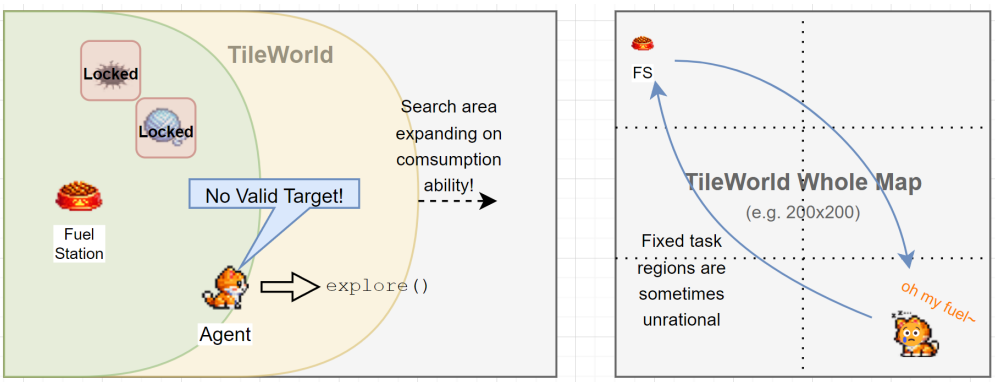


Figure 2: Dynamic Agents Working Area Indication

5 Result and Analysis

The following table summarizes the mean rewards achieved by our full system across the three parameter sets, averaged over 10 runs each:

	Parameter 1	Parameter 2	Parameter 3
G7 Agent System	1086.5	1397.2	2440.8

Table 1: Mean Rewards Achieved Across Different Environment Settings

Mean reward increases from Parameter Set 1 to 3, suggesting that higher object-generation intensity can outweigh the adverse effects of shorter lifetime and larger map size. The full-system performance serves as the reference point; attribution of gains is analyzed through the ablation results below.

6 Ablation Study

We investigated the contribution on the core designs of our agent system by disabling each of them and test on each configuration for 10 runs and take the mean value for comparison. One is the communication mechanism, we added a `CommunicationBlackhole` that absorbs all outgoing messages to remove communication layer while keeping other parts of the system exact. The other core design is each of our Agent's local policy and planner, we implemented a `MinimalAgent` that only has the basic mobility and survival functions provided by the `Group7AgentBase`, with a simple greedy policy to pick up the nearest tile and fill the nearest hole, without any of the additional logic in the original design.

Then we ran the ablation experiments with the settings and the results are shown in the table below.

Design Ablation	Parameter 1	Parameter 2	Parameter 3
Full System	**1086.5**	**1397.2**	**2440.8**
No Communication	402.2	371.7	61.4
BaseAgent Only	459.5	748.5	545.6

Table 2: Ablation Experiment Results (Mean Rewards over 10 runs)

Ablation reveals the source of improvement. Removing communication reduces reward by 63.0%, 73.4%, and 97.5% for Sets 1–3, respectively, indicating that shared situational awareness is critical in partially observable and highly dynamic settings. Replacing specialized policies with `AgentMinimal` also causes substantial drops (57.7%, 46.4%, 77.6%), showing that common infrastructure alone is insufficient. Overall, team performance depends on both protocol-level coordination and agent-level planning specialization.

7 Conclusion

This project developed a six-agent Tileworld system with a shared engineering backbone (`Group7AgentBase`) and agent-specific decision modules. The implemented framework unifies path planning, fuel-safe control, and a structured communication protocol (G7P2), while allowing each agent to introduce distinct planning and memory mechanisms.

Empirically, our results show that the complete system consistently outperforms ablated variants across all tested parameter sets. The ablation study further demonstrates that communication and policy specialization are complementary: removing either substantially harms final reward.

The current implementation still has limitations. Richer planners or exploration strategies can introduce nontrivial runtime overhead on large maps, and some systems rely heavily on SS message which have a high bandwidth cost. Future work should therefore target adaptive communication budgets that manages bandwidth-urgency trade-offs, and tighter integration between global coordination signals and local plan repair to improve robustness under extreme dynamism.

References

[1] Martha E. Pollack and Marc Ringuette. Introducing the tileworld: Experimentally evaluating agent architectures. In *Proceedings of the Eighth National Conference on Artificial Intelligence (AAAI-90)*, pages 183–189. AAAI Press, 1990.

[2] Michael Lees. A history of the tileworld agent testbed. Technical Report NOTTCS-WP-2002-1, School of Computer Science and Information Technology, University of Nottingham, March 2002.

[3] David G Kleinbaum and Mitchel Klein. *Survival Analysis: A Self-Learning Text*. Springer Science & Business Media, New York, 3rd edition, 2012.

Appendix

[AgentHanny]
Deriviation of the object survival probability estimation and the online MLE process for hazard estimation.

$$p_{alive} = e^{-\lambda \Delta t} \quad \in (0, 1]$$

Here, Δt is the estimated time of arrival from the current time stamp, and λ is a reflecting environment $hazard$ that affects the agent's expectation of an object still alive. This $harzard$ will be different across different experiment setups, such as world size and object creation distribution.

We model each observe on an object as a Bernoulli trial, with matching observation with possibility $P_{match} = \exp(-\lambda \Delta t)$ and a mismatch with possibility $P_{mismatch} = 1 - \exp(-\lambda \Delta t)$. Then we can use the maximum likelihood estimation (MLE) to find the best λ that fits the observation data. By computing the gradient of the negative log-likelihood $l(P)$, we can then perform the MLE via mini-batch stochastic gradient decent (SGD) with update rule as follows:

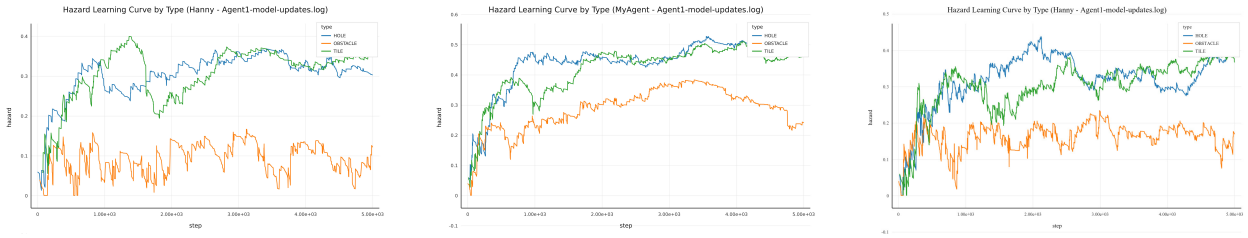
$$\text{Upon Match} \quad \frac{\partial \ell}{\partial \lambda} = -\Delta t \quad \Longrightarrow \quad \lambda \leftarrow \max\big(0, \lambda - \eta \Delta t\big)$$

$$\text{Upon MisMatch} \quad \frac{\partial \ell}{\partial \lambda} = \frac{\Delta t\, e^{-\lambda \Delta t}}{1 - e^{-\lambda \Delta t}} = \frac{\Delta t}{e^{\lambda \Delta t} - 1} \quad \Longrightarrow \quad \lambda \leftarrow \lambda + \eta \cdot \frac{\Delta t}{e^{\lambda \Delta t} - 1 + \epsilon}$$

By using learning rate schedule and gradient ceiling and clipping, the lambda is then converged to a stable range after 1000 steps on online training, as the curves shown in the figure below.

[AgentHanny]
Expected utility deriviation on a plan with multiple targets:

$$EU_{plan} = \sum_{pickup} p_{tile-alive} \cdot 0.20 + \sum_{fill} p_{hole-alive} \cdot 1.00 - 0.07 \cdot Distance - FuelPenalty$$



(a) λ vs Steps - Parameters 1 (b) λ vs Steps - Parameters 2 (c) λ vs Steps - Parameters 3

Figure 3: Hazard Figure Across Different Environment Settings (without smoothing).